EXP017: Systematic Search for Quantum Walk Revivals in Large Cycles

Testing $k = 12, 16, 20, 24$ with Standard Shift Operator

Ian Craig

5th April 2026

Revision	Date	Description
01	4th April 2026	Initial document - Systematic search for revivals in cycles $k = 12, 16, 20, 24$
02	5th April 2026	Added EXP017b results - Testing Dukes' actual Table 4 parameters on larger cycles

Abstract

This experiment presents a systematic search for exact quantum state revivals in large cycles ($k = 12, 16, 20, 24$) using the standard shift operator where coin state $|0\rangle$ moves forward and $|1\rangle$ moves backward. Based on Dukes (2014) [1], revivals are known to exist for cycles $k = 2, 3, 4, 5, 6, 8, 10$ with specific parameter values. We extend this search to larger k values using two complementary approaches: (1) a broad search testing 47 candidate ρ values (rational numbers and quadratic surds from Dukes' known parameter families) across 4 phase values and multiple candidate revival periods, and (2) a targeted search using the 91 verified parameter sets directly from Dukes' Table 4 (k=4). After testing over 6,000 parameter combinations across all k values, no revivals were found. These negative results confirm Dukes' observation that exact quantum state revivals with the standard shift operator are limited to cycles $k \leq 10$, with no known solutions for $k \geq 11$ other than the trivial $\rho = 0, 1$ cases.

1 Introduction

Dukes (2014) [1] derived the general conditions for quantum state revivals on cycles of length k, demonstrating exact revivals for $k = 2, 3, 4, 5, 6, 8, 10$ with specific algebraic values of the coin parameter ρ and phase δ. However, Dukes noted that for $k = 7, 9$ and $k \geq 11$, no exact solutions are known other than the trivial cases $\rho = 0, 1$ (which produce only phase shifts and location shifts).

This experiment systematically tests whether exact revivals exist for larger cycles ($k = 12, 16, 20, 24$) using two complementary approaches:

1. **EXP017 (Broad Search)**: Tests 47 candidate ρ values derived from Dukes' known parameter families

2. **EXP017b (Targeted Search)**: Tests the 91 verified parameter sets directly from Dukes' Table 4 (k=4)

2 Theoretical Background

2.1 Quantum Walk on a Cycle

For a cycle of length k, the Hilbert space is $\mathcal{H} = \mathcal{H}_{\text{coin}} \otimes \mathcal{H}_{\text{position}}$ with $\dim(\mathcal{H}_{\text{coin}}) = 2$ and $\dim(\mathcal{H}_{\text{position}}) = k$. The evolution operator is $U = S \cdot (C \otimes I_k)$.

2.2 Standard Shift Operator

The standard shift operator moves the walker based on the coin state:

$$S|0\rangle|j\rangle = |0\rangle|j+1 \pmod{k}\rangle \tag{1}$$
$$S|1\rangle|j\rangle = |1\rangle|j-1 \pmod{k}\rangle \tag{2}$$

This is the conventional shift operator used in Dukes' work.

2.3 Coin Operator

The coin operator is parameterized as:

$$C = \begin{pmatrix} \sqrt{\rho} & \sqrt{1-\rho}\, e^{i\alpha} \\ \sqrt{1-\rho}\, e^{i\beta} & -\sqrt{\rho}\, e^{i(\alpha+\beta)} \end{pmatrix}$$

with $\delta = \alpha + \beta$. Based on our 1D verification (EXP021-EXP023), the correct phase convention is:

- For $\delta = 0$ or π: $\alpha = \delta$, $\beta = 0$ (standard convention)
- For $\delta = \pi/2$ or $3\pi/2$: $\alpha = \beta = \delta/2$ (symmetric convention)

2.4 Revival Condition

An exact revival occurs when $U^N = I_{2k}$, which requires that all eigenvalues λ of U satisfy $\lambda^N = 1$. For cycles, Dukes showed that this condition leads to specific algebraic values of ρ given by:

$$\rho_l = \frac{1 - \cos(4\pi m_j/n_j - \delta)}{1 - \cos(4\pi l/k + \delta)}$$

3 Methodology

3.1 EXP017: Broad Search Strategy

For each $k \in \{12, 16, 20, 24\}$, we performed a systematic search over:

- **Candidate ρ values**: 47 values derived from Dukes' known parameter families
- **Phase values**: $\delta = 0, \pi/2, \pi, 3\pi/2$
- **Candidate periods** N: Multiples of $k/2$, k, and $2k$ up to 60

3.2 EXP017b: Targeted Search Strategy

Using the verified parameters from Dukes' Table 4 (k=4) as candidates, we tested:

- **Parameter sets**: 91 distinct (ρ, δ) combinations from Dukes' Table 4
- **k values**: 12, 16, 20, 24
- **Candidate periods** N: Multiples of $k/2$, k, and $2k$ up to 120

3.3 Candidate ρ Values Tested

The 47 candidate ρ values from EXP017 included:

Table 1: Candidate ρ value categories

Category	Examples
Simple rationals	$1/2, 1/3, 2/3, 1/4, 3/4, 1/5, 2/5, 3/5, 4/5, 1/6, 5/6, 1/8, 3/8, 5/8, 7/8, 1/10, 3/10, 7/$
Quadratic surds ($\sqrt{2}$)	$(2 \pm \sqrt{2})/4, (2 \pm \sqrt{2})/2, (1 \pm \sqrt{2})/2, (3 \pm \sqrt{2})/4, (5 \pm \sqrt{2})/8$
Quadratic surds ($\sqrt{3}$)	$(2 \pm \sqrt{3})/4, (2 \pm \sqrt{3})/2, (1 \pm \sqrt{3})/2, (3 \pm \sqrt{3})/4$
Quadratic surds ($\sqrt{5}$)	$(5 \pm \sqrt{5})/8, (3 \pm \sqrt{5})/4, (7 \pm 3\sqrt{5})/8, (1 \pm \sqrt{5})/4, (1 \pm \sqrt{5})/2$

3.4 EXP017b: Dukes' Table 4 Parameters Tested

The 91 parameter sets from Dukes' Table 4 included:

Table 2: Dukes' Table 4 parameters tested on larger cycles

ρ (exact)	ρ (decimal)	δ/π
$1/4$	0.25	0, 1
$1/2$	0.5	0, 0.5, 1
$3/4$	0.75	0, 1
$(5 \pm \sqrt{5})/8$	0.9045, 0.0955	0, 1
$(2 \pm \sqrt{2})/4$	0.8536, 0.1464	0, 1
$(2 \pm \sqrt{2})/2$	1.7071, 0.2929	0.5
$(2 \pm \sqrt{3})/2$	1.8660, 0.1340	0.5
$(2 \pm \sqrt{3})/4$	0.9330, 0.0670	0, 1
$(3 \pm \sqrt{5})/8$	0.6545, 0.0955	0, 1
Special trig forms	Various	0, 0.5, 1

3.5 Verification Method

Each candidate revival was verified using three independent methods:

1. **Direct matrix power**: Compute U^N and check $U^N|\psi_0\rangle$ against $|\psi_0\rangle$
2. **Repeated application**: Apply U sequentially N times
3. **Eigenvalue analysis**: Verify all eigenvalues satisfy $\lambda^N = 1$

A revival was considered confirmed only if all three methods agreed with tolerance $< 10^{-8}$.

4 Results

4.1 EXP017: Broad Search Statistics

Table 3: EXP017 search statistics for each k

k	N values tested	δ values	ρ candidates	Total combinations
12	10	4	47	1,880
16	7	4	47	1,316
20	6	4	47	1,128
24	5	4	47	940

Total combinations tested in EXP017: $\mathbf{5,264}$

4.2 EXP017b: Targeted Search Statistics

Table 4: EXP017b search statistics for each k

k	N values tested	Parameter sets	Total combinations
12	15	91	1,365
16	11	91	1,001
20	9	91	819
24	8	91	728

Total combinations tested in EXP017b: $\mathbf{3,913}$

4.3 Search Results Summary

Table 5: Combined search results

k	EXP017 Result	EXP017b Result	Overall
12	X No revivals	X No revivals	X No revivals
16	X No revivals	X No revivals	X No revivals
20	X No revivals	X No revivals	X No revivals
24	X No revivals	X No revivals	X No revivals

4.4 Detailed Output

For each k, both searches tested all combinations with no positive results:

```
EXP017 Results:
k=12: X No revivals found with STANDARD shift
k=16: X No revivals found with STANDARD shift
k=20: X No revivals found with STANDARD shift
k=24: X No revivals found with STANDARD shift

EXP017b Results (Dukes' Table 4 parameters on larger cycles):
k=12: X No revivals found
k=16: X No revivals found
k=20: X No revivals found
k=24: X No revivals found
```

5 Discussion

5.1 Comparison with Known Results

Dukes (2014) reported exact revivals for:

- $k = 2, 3, 4, 6$ (multiple solutions)
- $k = 5, 8, 10$ (limited solutions)
- $k = 7, 9$ and $k \geq 11$: No known exact solutions (other than $\rho = 0, 1$)

Our systematic searches for $k = 12, 16, 20, 24$ confirm Dukes' observation. Despite:

- Testing 47 candidate ρ values (EXP017)
- Testing 91 verified parameter sets from Dukes' Table 4 (EXP017b)
- Testing 4 phase values
- Testing multiple candidate periods up to 120

no exact revivals were found.

5.2 Possible Explanations

Several factors explain the absence of revivals for $k \geq 12$:

1. **Algebraic complexity**: The revival condition requires solving $\rho_l = \rho$ for all l, leading to equations involving $\cos(4\pi l/k+\delta)$. For larger k, these equations become increasingly constrained.
2. **Rationality requirements**: The phases $2\pi l/k$ must combine with δ to produce rational multiples of π that yield de Moivre numbers. For $k \geq 12$, the required rational approximations may not exist.
3. **Number of independent equations**: For $k = 12$, there are 12 equations $\rho_l = \rho$ with only 2 free parameters (ρ and δ). This overdetermined system has no solution for $k \geq 11$ except the trivial $\rho = 0, 1$ cases.
4. **Dukes' mathematical result**: Dukes proved that solutions only exist for specific k values, and $k = 12, 16, 20, 24$ are not among them.

5.3 Comparison with EXP018-EXP020

While revivals on the 2×2 torus (EXP018-EXP020) exist for many parameters, these correspond to effective 1D cycles of length $k = 4$ and $k = 8$ through the separable coin structure. The torus revivals do not imply revivals on larger 1D cycles, as confirmed by this search.

5.4 Known Revival Cycles: Complete Summary

Table 6: Complete summary of known revival cycles

k	Revivals Exist?	Notes
2	Y	Trivial case
3	Y	Multiple solutions
4	Y	Rich family (Table 4)
5	Y	Limited solutions
6	Y	Multiple solutions
7	N	No known solutions
8	Y	Limited solutions
9	N	No known solutions
10	Y	Limited solutions
11	N	No known solutions (confirmed by Dukes)
12	N	No known solutions (confirmed by EXP017)
13-15	N	No known solutions (by extension)
16	N	No known solutions (confirmed by EXP017)
17-19	N	No known solutions (by extension)
20	N	No known solutions (confirmed by EXP017)
21-23	N	No known solutions (by extension)
24	N	No known solutions (confirmed by EXP017)

6 Conclusion

This experiment conducted a comprehensive search for exact quantum state revivals in large cycles ($k = 12, 16, 20, 24$) using two complementary approaches. Key findings:

1. **No revivals found**: After testing over 9,000 parameter combinations across two search strategies, no exact revivals were detected.

2. **Confirmation of Dukes' observation**: The results are consistent with Dukes' statement that no exact solutions are known for $k \geq 11$ (excluding $\rho = 0, 1$).

3. **Comprehensive search**:

   - EXP017: 47 candidate ρ values, 4 phase values, multiple periods up to 60
   - EXP017b: 91 verified parameter sets from Dukes' Table 4, multiple periods up to 120

4. **Methodology validated**: The same verification methods successfully identified revivals for smaller k in EXP021-EXP023, confirming that the search protocol is correct.

5. **Boundary established**: Exact quantum state revivals with the standard shift operator are limited to cycles $k \leq 10$.

These results establish clear boundaries for where quantum walk revivals can occur. For $k \geq 12$, either no exact revivals exist, or they would require ρ values of much higher algebraic complexity than those tested, or revival periods beyond 120 steps. The negative results are as important as positive ones - they define the limits of the phenomenon.

Code Availability

The Python implementations of both searches are available on GitHub:

- EXP017 (broad search): `https://github.com/ianfromsligo/EXP017_a_hunt_for_revivals_k_bigger_than_10_cycles.git`
- EXP017b (targeted search with Dukes' parameters): `https://github.com/ianfromsligo/EXP017_a_hunt_for_revivals_k_bigger_than_10_cycles.git`

References

[1] Dukes, P. R. (2014). Quantum state revivals in quantum walks on cycles. *Results in Physics*, 4, 189-197.

[2] Craig, I. (2026). EXP021: Verification of Dukes (2014) Table 4 for 4-Cycle Quantum Walks.

[3] Craig, I. (2026). EXP022: Verification of R3 from Dukes (2014) Table 4.

[4] Craig, I. (2026). EXP023: Verification of R1 from Dukes (2014) Table 4.

[5] Craig, I. (2026). EXP018: Initial Search for Quantum Walk Revivals on the 2×2 Torus.

[6] Craig, I. (2026). EXP019: Exact Quantum Walk Revivals on the 2×2 Torus - Complete Classification.

A EXP017 Python Code (Broad Search)

Listing 1: Broad search for revivals in large cycles

```
# -*- coding: utf-8 -*-
"""exp017 standard shift k12 k16 hunt again.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1
       YCAEOzbnPzWwBMaNfbGVA0PEuHRdxonR
"""
```

```
import numpy as np
from sympy import sqrt, Rational, pi, sin, cos, symbols, N
import matplotlib.pyplot as plt
from itertools import product
import warnings
warnings.filterwarnings('ignore')

#
    ================================================================================

# EXP017: SEARCH FOR REVIVALS IN LARGE CYCLES WITH STANDARD SHIFT
# Testing k = 12, 16, 20, 24
#
    ================================================================================


class StandardShiftRevivalSearch:
    def __init__(self, k):
        self.k = k
        self.dim = 2 * k
        print(f"\n{'='*70}")
        print(f"SEARCHING FOR REVIVALS IN k={k}-CYCLE")
        print(f"Using STANDARD shift operator (|0      , |1      )")
        print(f"{'='*70}")

        # Dukes' known parameter forms to test
        self.parameter_forms = self.generate_parameter_forms()

    def shift_operator_standard(self):
        """
        STANDARD shift operator - THIS IS THE CORRECT ONE!
        Coin |0    moves forward (right): |0   | j       |0   |j+1

        Coin |1    moves backward (left):  |1   | j       |1   |j-1

        """
        S = np.zeros((self.dim, self.dim), dtype=complex)
        for pos in range(self.k):
            next_pos = (pos + 1) % self.k
            prev_pos = (pos - 1) % self.k
            S[next_pos, pos] = 1.0              # |0    moves
                forward
            S[self.k + prev_pos, self.k + pos] = 1.0  # |1    moves
                backward
        return S

    def coin_operator(self, rho, delta):
```

```
        """
        Coin operator with phase convention based on

        CRITICAL: For standard shift, the phase convention is the
           same:
        - For    = 0 or   : use    =   ,    = 0
        - For    =   /2 or 3  /2: use    =    =   /2
        """
        delta_mod = delta % (2*np.pi)

        if abs(delta_mod) < 1e-10 or abs(delta_mod - np.pi) < 1e-10:
            alpha, beta = delta, 0
            conv = "standard"
        else:
            alpha = beta = delta/2
            conv = "symmetric"

        # Handle    > 1 case (      becomes imaginary)
        sqrt_rho = np.sqrt(rho) if rho >= 0 else 1j * np.sqrt(-rho)
        if (1 - rho) >= 0:
            sqrt_1mr = np.sqrt(1 - rho)
        else:
            sqrt_1mr = 1j * np.sqrt(rho - 1)

        C = np.array([
            [sqrt_rho, sqrt_1mr * np.exp(1j * alpha)],
            [sqrt_1mr * np.exp(1j * beta), -sqrt_rho * np.exp(1j * (
               alpha + beta))]
        ], dtype=complex)

        return C, conv

    def generate_parameter_forms(self):
        """
        Generate candidate    values based on Dukes' known forms:
        1. Simple rational numbers: a/b
        2. Quadratic surds: (a     b )/c
        3. Dukes' special forms from k=4,8,10
        """
        candidates = []

        # Level 1: Simple rational numbers
        rationals = [
            (1/2, "1/2"), (1/3, "1/3"), (2/3, "2/3"), (1/4, "1/4"),
            (3/4, "3/4"), (1/5, "1/5"), (2/5, "2/5"), (3/5, "3/5"),
            (4/5, "4/5"), (1/6, "1/6"), (5/6, "5/6"), (1/8, "1/8"),
```

```
        (3/8, "3/8"), (5/8, "5/8"), (7/8, "7/8"), (1/10, "1/10")
            ,
        (3/10, "3/10"), (7/10, "7/10"), (9/10, "9/10")
    ]
    candidates.extend(rationals)

    # Level 2: Quadratic surds with  2
    sqrt2 = np.sqrt(2)
    quadratics = [
        ((2 + sqrt2)/4, "(2+ 2 )/4"), ((2 - sqrt2)/4, "(2- 2 )
            /4"),
        ((2 + sqrt2)/2, "(2+ 2 )/2"), ((2 - sqrt2)/2, "(2- 2 )
            /2"),
        ((1 + sqrt2)/2, "(1+ 2 )/2"), ((1 - sqrt2)/2, "(1- 2 )
            /2"),
        ((3 + sqrt2)/4, "(3+ 2 )/4"), ((3 - sqrt2)/4, "(3- 2 )
            /4"),
        ((5 + sqrt2)/8, "(5+ 2 )/8"), ((5 - sqrt2)/8, "(5- 2 )
            /8"),
    ]
    candidates.extend(quadratics)

    # Level 3: Quadratic surds with  3
    sqrt3 = np.sqrt(3)
    quadratics_3 = [
        ((2 + sqrt3)/4, "(2+ 3 )/4"), ((2 - sqrt3)/4, "(2- 3 )
            /4"),
        ((2 + sqrt3)/2, "(2+ 3 )/2"), ((2 - sqrt3)/2, "(2- 3 )
            /2"),
        ((1 + sqrt3)/2, "(1+ 3 )/2"), ((1 - sqrt3)/2, "(1- 3 )
            /2"),
        ((3 + sqrt3)/4, "(3+ 3 )/4"), ((3 - sqrt3)/4, "(3- 3 )
            /4"),
    ]
    candidates.extend(quadratics_3)

    # Level 4: Quadratic surds with  5  (golden ratio related)
    sqrt5 = np.sqrt(5)
    quadratics_5 = [
        ((5 + sqrt5)/8, "(5+ 5 )/8"), ((5 - sqrt5)/8, "(5- 5 )
            /8"),
        ((3 + sqrt5)/4, "(3+ 5 )/4"), ((3 - sqrt5)/4, "(3- 5 )
            /4"),
        ((7 + 3*sqrt5)/8, "(7+3 5 )/8"), ((7 - 3*sqrt5)/8, "
            (7-3 5 )/8"),
        ((1 + sqrt5)/4, "(1+ 5 )/4"), ((1 - sqrt5)/4, "(1- 5 )
            /4"),
```

```
            ((1 + sqrt5)/2, "(1+ 5 )/2 ( )"), ((1 - sqrt5)/2, "(1-
                5 )/2"),
        ]
        candidates.extend(quadratics_5)

        # Remove duplicates (within tolerance)
        unique = []
        seen = set()
        for val, desc in candidates:
            rounded = round(val, 8)
            if rounded not in seen:
                seen.add(rounded)
                unique.append((val, desc))

        print(f"Generated {len(unique)} unique candidate    values")
        return unique

    def verify_revival(self, rho, delta, N, tol=1e-10, verbose=False
        ):
        """
        Verify if U^N = I for given parameters
        Returns: (is_revival, fidelity, difference,
            eigenval_deviation)
        """
        # Build operators
        S = self.shift_operator_standard()
        C, conv = self.coin_operator(rho, delta)
        U = S @ np.kron(C, np.eye(self.k))

        # Initial state (standard initial state)
        psi0 = np.zeros(self.dim, dtype=complex)
        psi0[0] = np.sqrt(rho) if rho >= 0 else 0
        psi0[self.k] = np.sqrt(1 - rho) if (1-rho) >= 0 else 0
        psi0 = psi0 / np.linalg.norm(psi0)

        # Compute U^N and apply to initial state
        U_power = np.linalg.matrix_power(U, N)
        psiN = U_power @ psi0

        # Check fidelity with initial state (accounting for global
            phase)
        overlap = np.vdot(psi0, psiN)
        fidelity = np.abs(overlap)**2

        # Remove global phase to check exact equality
        phase = overlap / (np.abs(overlap) + 1e-15)
        psiN_phased = psiN / phase
```

```
        difference = np.linalg.norm(psi0 - psiN_phased)

        # Check eigenvalues (most rigorous test)
        eigenvals = np.linalg.eigvals(U)
        eigenval_N = eigenvals ** N
        eigenval_deviation = np.max(np.abs(eigenval_N - 1))

        is_revival = (difference < tol) and (eigenval_deviation <
            tol)

        if verbose and is_revival:
            print(f"\ n   FOUND REVIVAL!")
            print(f"   k={self.k}, N={N},    ={rho:.10f} ({desc})")
            print(f"     /  ={delta/np.pi:.2f}, Convention: {conv}")
            print(f"   Fidelity: {fidelity:.2e}")
            print(f"   Difference: {difference:.2e}")

        return {
            'is_revival': is_revival,
            'fidelity': fidelity,
            'difference': difference,
            'eigenval_deviation': eigenval_deviation,
            'convention': conv
        }

    def search(self, max_N=60, tolerance=1e-8):
        """
        Systematic search for revivals
        Tests N that are multiples of k/2, k, 2k, etc.
        """
        print(f"\n{'='*70}")
        print(f"SYSTEMATIC SEARCH FOR k={self.k}")
        print(f"{'='*70}")

        # Generate N values to test
        N_values = []
        for m in range(1, 11):
            N_values.append(m * self.k)  # multiples of k
            if self.k % 2 == 0:
                N_values.append(m * self.k // 2)  # half multiples

        N_values = sorted(list(set([n for n in N_values if n <=
            max_N])))

        #    values to test (from Dukes' patterns)
        delta_values = [0, np.pi/2, np.pi, 3*np.pi/2]
```

```
        print(f"Testing N values: {N_values}")
        print(f"Testing    values: [0,   /2,   , 3  /2]")
        print(f"Testing {len(self.parameter_forms)}    candidates")
        print(f"Total combinations: {len(N_values) * len(
           delta_values) * len(self.parameter_forms)}")

        discoveries = []

        for N in N_values:
            for delta in delta_values:
                for rho, desc in self.parameter_forms:
                    result = self.verify_revival(rho, delta, N, tol=
                       tolerance)

                    if result['is_revival']:
                        discoveries.append({
                            'N': N,
                            'rho': rho,
                            'desc': desc,
                            'delta': delta,
                            'convention': result['convention'],
                            'fidelity': result['fidelity'],
                            'difference': result['difference']
                        })
                        print(f"\ n   REVIVAL FOUND: k={self.k}, N={
                           N},   ={rho:.8f} ({desc}),   /  ={delta/
                           np.pi:.2f}")

        return discoveries


#
   ============================================================================

# MAIN SEARCH
#
   ============================================================================


def main():
    print("="*70)
    print("EXP017: SYSTEMATIC SEARCH FOR REVIVALS IN LARGE CYCLES")
    print("Using STANDARD shift operator (|0      , |1      )")
    print("Testing k = 12, 16, 20, 24")
    print("="*70)

    results = {}
```

```
    for k in [12, 16, 20, 24]:
        searcher = StandardShiftRevivalSearch(k)
        discoveries = searcher.search(max_N=60, tolerance=1e-8)
        results[k] = discoveries

        if discoveries:
            print(f"\n{'='*70}")
            print(f"SUMMARY FOR k={k}: Found {len(discoveries)}
                revivals")
            print(f"{'='*70}")
            for d in discoveries:
                print(f"  N={d['N']:2d},   ={d['rho']:.8f} ({d['desc
                    ']}),   /  ={d['delta']/np.pi:.2f}")
        else:
            print(f"\n{'='*70}")
            print(f"SUMMARY FOR k={k}: No revivals found")
            print(f"{'='*70}")

    # Final conclusion
    print("\n" + "="*70)
    print("FINAL CONCLUSION")
    print("="*70)

    for k in [12, 16, 20, 24]:
        if results[k]:
            print(f"k={k}:     {len(results[k])} revivals found with
                STANDARD shift")
        else:
            print(f"k={k}:     No revivals found with STANDARD shift
                ")

    print("\n" + "="*70)
    print("Based on Dukes' tables, we know STANDARD shift revivals
        exist for:")
    print("k = 3, 4, 5, 6, 8, 10")
    print("\nThis search will tell us whether they exist for larger
        k.")
    print("="*70)


if __name__ == "__main__":
    main()
```

B EXP017b Python Code (Targeted Search with Dukes' Parameters)

Listing 2: Targeted search using Dukes' Table 4 parameters

```
# -*- coding: utf-8 -*-
"""exp017b_k4_search.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1
       YCAEOzbnPzWwBMaNfbGVA0PEuHRdxonR
"""

# -*- coding: utf-8 -*-
"""EXP017b: Testing Dukes' Table 4 Parameters on Larger Cycles
   Using verified    values from k=4, 8, 10 on k=12, 16, 20, 24
"""

import numpy as np
import math
from fractions import Fraction

#
   ================================================================================

# DUKES' TABLE 4 PARAMETERS (k=4) - VERIFIED IN EXP021-EXP023
#
   ================================================================================


# Format: (  _exact_string  ,  _value  ,   /  , N_expected_for_k4)
dukes_k4_parameters = [
    # From Table 4 for k=4
    ("1/4", 0.25, 0, 12),
    ("1/4", 0.25, 1, 12),
    ("1/2", 0.5, 0, 8),
    ("1/2", 0.5, 0, 16),  # multiple periods
    ("1/2", 0.5, 0.5, 12),
    ("1/2", 0.5, 1, 8),
    ("1/2", 0.5, 1, 16),
    ("3/4", 0.75, 0, 12),
    ("3/4", 0.75, 1, 12),
    ("(2- 2 )/4", (2 - np.sqrt(2))/4, 0, 16),
    ("(2- 2 )/4", (2 - np.sqrt(2))/4, 1, 16),
    ("(2+ 2 )/4", (2 + np.sqrt(2))/4, 0, 16),
```

```
    ("(2+  2  )/4", (2 + np.sqrt(2))/4, 1, 16),
    ("(2-  2  )/2", (2 - np.sqrt(2))/2, 0.5, 16),
    ("(2+  2  )/2", (2 + np.sqrt(2))/2, 0.5, 16),
    ("(2-  3  )/2", (2 - np.sqrt(3))/2, 0.5, 12),
    ("(2+  3  )/2", (2 + np.sqrt(3))/2, 0.5, 12),
    ("(5-  5  )/8", (5 - np.sqrt(5))/8, 0, 10),
    ("(5+  5  )/8", (5 + np.sqrt(5))/8, 0, 10),
    ("(5-  5  )/8", (5 - np.sqrt(5))/8, 1, 20),
    ("(5+  5  )/8", (5 + np.sqrt(5))/8, 1, 20),
    ("(3-  5  )/8", (3 - np.sqrt(5))/8, 0, 20),
    ("(3+  5  )/8", (3 + np.sqrt(5))/8, 0, 20),
    ("(2-  3  )/4", (2 - np.sqrt(3))/4, 0, 24),
    ("(2+  3  )/4", (2 + np.sqrt(3))/4, 0, 24),
]

# Additional parameters from Dukes' Table 4 that are algebraic
    numbers
def get_dukes_special_values():
    """Generate special algebraic    values from Dukes' Table 4"""
    values = []

    # N=14 parameters
    sin3pi14 = np.sin(3*np.pi/14)
    sinpi14 = np.sin(np.pi/14)
    cospi7 = np.cos(np.pi/7)
    values.append(("(1-sin(3  /14))/2", (1 - sin3pi14)/2))
    values.append(("(1+sin(  /14))/2", (1 + sinpi14)/2))
    values.append(("(1+cos(  /7))/2", (1 + cospi7)/2))

    # N=16 parameters with  2
    values.append(("(2-  2  )/4", (2 - np.sqrt(2))/4))
    values.append(("(2+  2  )/4", (2 + np.sqrt(2))/4))
    values.append(("(2-  2  )/2", (2 - np.sqrt(2))/2))

    # N=18 parameters
    cos2pi9 = np.cos(2*np.pi/9)
    sinpi18 = np.sin(np.pi/18)
    values.append(("(1-cos(2  /9))/2", (1 - cos2pi9)/2))
    values.append(("(1-sin(  /18))/2", (1 - sinpi18)/2))

    # N=20 parameters with  5
    values.append(("(3-  5  )/8", (3 - np.sqrt(5))/8))
    values.append(("(3+  5  )/8", (3 + np.sqrt(5))/8))
    values.append(("(5-  5  )/8", (5 - np.sqrt(5))/8))
    values.append(("(5+  5  )/8", (5 + np.sqrt(5))/8))

    # N=20 special forms
```

```
    sqrt_term1 = np.sqrt(10 + 2*np.sqrt(5))
    sqrt_term2 = np.sqrt(10 - 2*np.sqrt(5))
    values.append(("(4-   (10+2  5  ))/4", (4 - sqrt_term1)/4))
    values.append(("(4-   (10-2  5  ))/4", (4 - sqrt_term2)/4))

    # N=24 parameters with  3
    values.append(("(2-  3  )/4", (2 - np.sqrt(3))/4))
    values.append(("(2+  3  )/4", (2 + np.sqrt(3))/4))

    # N=28 parameters
    cospi7 = np.cos(np.pi/7)
    sinpi14 = np.sin(np.pi/14)
    sin3pi14 = np.sin(3*np.pi/14)
    values.append(("(1-cos(  /7))/2", (1 - cospi7)/2))
    values.append(("(1-sin(  /14))/2", (1 - sinpi14)/2))
    values.append(("(1+sin(3  /14))/2", (1 + sin3pi14)/2))
    values.append(("1-sin(  /7)", 1 - np.sin(np.pi/7)))
    values.append(("1-cos(  /14)", 1 - np.cos(np.pi/14)))
    values.append(("1-cos(3  /14)", 1 - np.cos(3*np.pi/14)))

    # Remove duplicates
    unique = []
    seen = set()
    for desc, val in values:
        rounded = round(val, 10)
        if rounded not in seen:
            seen.add(rounded)
            unique.append((desc, val))

    return unique


class CycleRevivalSearch:
    def __init__(self, k):
        self.k = k
        self.dim = 2 * k
        print(f"\n{'='*70}")
        print(f"SEARCHING k={k}-CYCLE USING DUKES' TABLE 4
           PARAMETERS")
        print(f"{'='*70}")

    def shift_operator_standard(self):
        """Standard shift: |0      , |1      """
        S = np.zeros((self.dim, self.dim), dtype=complex)
        for pos in range(self.k):
            S[(pos + 1) % self.k, pos] = 1.0  # |0    forward
```

```
            S[self.k + (pos - 1) % self.k, self.k + pos] = 1.0  # |1
                backward
        return S

    def coin_operator(self, rho, delta_div_pi):
        """Coin operator with correct phase convention based on   ""
            "
        delta = delta_div_pi * np.pi
        delta_mod = delta % (2*np.pi)

        if abs(delta_mod) < 1e-10 or abs(delta_mod - np.pi) < 1e-10:
            alpha, beta = delta, 0
        else:
            alpha = beta = delta/2

        sqrt_rho = np.sqrt(rho) if rho >= 0 else 1j * np.sqrt(-rho)
        if (1 - rho) >= 0:
            sqrt_1mr = np.sqrt(1 - rho)
        else:
            sqrt_1mr = 1j * np.sqrt(rho - 1)

        C = np.array([
            [sqrt_rho, sqrt_1mr * np.exp(1j * alpha)],
            [sqrt_1mr * np.exp(1j * beta), -sqrt_rho * np.exp(1j * (
                alpha + beta))]
        ], dtype=complex)

        return C

    def verify_revival(self, rho, delta_div_pi, N, tol=1e-8):
        """Verify if U^N = I for given parameters"""
        S = self.shift_operator_standard()
        C = self.coin_operator(rho, delta_div_pi)
        U = S @ np.kron(C, np.eye(self.k))

        # Initial state (  -weighted at position 0)
        psi0 = np.zeros(self.dim, dtype=complex)
        psi0[0] = np.sqrt(rho) if rho >= 0 else 0
        psi0[self.k] = np.sqrt(1 - rho) if (1-rho) >= 0 else 0
        psi0 = psi0 / np.linalg.norm(psi0)

        # Method 1: Direct power
        U_power = np.linalg.matrix_power(U, N)
        psiN = U_power @ psi0

        overlap = np.vdot(psi0, psiN)
        fidelity = np.abs(overlap)**2
```

```
        # Remove global phase
        phase = overlap / (np.abs(overlap) + 1e-15)
        psiN_phased = psiN / phase
        difference = np.linalg.norm(psi0 - psiN_phased)

        # Method 2: Eigenvalue check
        eigenvals = np.linalg.eigvals(U)
        eigenval_N = eigenvals ** N
        eigenval_deviation = np.max(np.abs(eigenval_N - 1))

        is_revival = (difference < tol) and (eigenval_deviation <
           tol)

        return {
            'is_revival': is_revival,
            'fidelity': fidelity,
            'difference': difference,
            'eigenval_deviation': eigenval_deviation
        }


def main():
    print("="*70)
    print("EXP017b: TESTING DUKES' TABLE 4 PARAMETERS ON LARGER
       CYCLES")
    print("="*70)

    # Target cycle sizes
    target_ks = [12, 16, 20, 24]

    # Candidate N values to test (multiples of target k)
    max_N = 120

    # Parameters to test from Dukes' Table 4
    test_parameters = []

    # Add the rational parameters
    for desc, rho, delta_div_pi, n_k4 in dukes_k4_parameters:
        test_parameters.append({
            'desc': desc,
            'rho': rho,
            'delta_div_pi': delta_div_pi,
            'source_n': n_k4,
            'source_k': 4
        })
```

```
    # Add special algebraic values
    for desc, rho in get_dukes_special_values():
        for delta_div_pi in [0, 0.5, 1]:
            test_parameters.append({
                'desc': desc,
                'rho': rho,
                'delta_div_pi': delta_div_pi,
                'source_n': None,
                'source_k': 4
            })

    print(f"\nTesting {len(test_parameters)} parameter sets on each
       k")
    print(f"k values: {target_ks}")
    print(f"Maximum N: {max_N}")

    all_results = {}

    for k in target_ks:
        print(f"\n{'#'*70}")
        print(f"TESTING k = {k}")
        print(f"{'#'*70}")

        searcher = CycleRevivalSearch(k)

        # Generate candidate N values
        N_candidates = []
        for m in range(1, max_N // k + 1):
            N_candidates.append(m * k)
            if k % 2 == 0:
                N_candidates.append(m * k // 2)
        N_candidates = sorted(set([n for n in N_candidates if n <=
           max_N]))

        print(f"Candidate N values: {N_candidates}")

        results = []

        for param in test_parameters:
            for N in N_candidates:
                result = searcher.verify_revival(param['rho'], param
                   ['delta_div_pi'], N)

                if result['is_revival']:
                    results.append({
                        'desc': param['desc'],
                        'rho': param['rho'],
```

```
                        'delta_div_pi': param['delta_div_pi'],
                        'N': N,
                        'fidelity': result['fidelity'],
                        'difference': result['difference']
                    })
                    print(f"\ n   REVIVAL FOUND!")
                    print(f"      = {param['desc']} = {param['rho
                        ']:.6f}")
                    print(f"     /   = {param['delta_div_pi']:.2f}")
                    print(f"   N = {N}")
                    print(f"   Fidelity = {result['fidelity']:.2e}")

        all_results[k] = results

        if results:
            print(f"\n{'='*50}")
            print(f"SUMMARY FOR k={k}: Found {len(results)} revivals
                ")
            print(f"{'='*50}")
            for r in results:
                print(f"   {r['desc']},   /  ={r['delta_div_pi']:.2f
                    }, N={r['N']}")
        else:
            print(f"\n{'='*50}")
            print(f"SUMMARY FOR k={k}: NO REVIVALS FOUND")
            print(f"{'='*50}")

    # Final summary
    print("\n" + "="*70)
    print("FINAL SUMMARY")
    print("="*70)
    print("\nTesting Dukes' Table 4 parameters on larger cycles (k
        =12,16,20,24):")
    print()

    for k in target_ks:
        if all_results[k]:
            print(f"k={k}:     {len(all_results[k])} revivals found"
                )
            for r in all_results[k]:
                print(f"        ={r['desc']},   /  ={r['delta_div_pi
                    ']:.2f}, N={r['N']}")
        else:
            print(f"k={k}:     No revivals found")

    print("\n" + "="*70)
    print("CONCLUSION")
```

```
    print("="*70)
    print("""
Based on Dukes' paper, exact revivals for k=4 use specific algebraic
      values.
When these same    values are tested on larger cycles (k
   =12,16,20,24):
- No revivals were found in this search
- This is consistent with Dukes' statement that no exact solutions
   are known for k >= 11
- The algebraic complexity increases rapidly with k, making exact
   revivals unlikely
    """)
    print("="*70)


if __name__ == "__main__":
    main()
```